Звіт про виконання лабораторної роботи №4

Тема: Модульне ядро, Clean Architecture та Dependency Injection

Дисципліна: Проєктування архітектури, специфікація та стандарти інженерії
Заклад: Херсонський національний технічний університет
Студент: не надано
Група: не надано
Тег у репозиторії: `lr4`
Посилання на репозиторій: локальний Git-репозиторій у папці проєкту; віддалений URL не надано
Стек: Python 3.10+, unittest, unittest.mock, стандартна бібліотека

Предметна область: E-Commerce Cloud Platform, компонент Order Service. Сценарій створення замовлення продовжує `CreateOrder` та `POST /orders` із контрактів лабораторної №3. Поля й обмеження звірено з локальними файлами `lab3_ecommerce_architecture/api/openapi.yaml` та `api/service.proto`. Їх точні копії включено в contracts_lab3 (contracts_lab3/README.md). Матеріали ЛР №1-2 окремо не перевірялися; зв'язок із попередньою роботою підтверджено безпосередньо контрактом ЛР №3, а не припущенням про нову систему.

1. Шари і правило залежностей

Мета: реалізувати модульне ядро сервісу замовлень, ізолювавши бізнес-правила від способу зберігання, HTTP та DI-контейнера.

```
src/
  Domain/
    order.py            Order, OrderItem, ProductSnapshot, Money,
                        Address, OrderStatus, DomainError
    ports.py            OrderRepository, RepositoryError
  Application/
    create_order.py     CreateOrder, DTO, Clock, IdGenerator
  Infrastructure/
    adapters.py         InMemoryOrderRepository, MemoryOrderStore,
                        SystemClock, UuidGenerator
  WebAPI/
    bootstrap.py        композиційний корінь і HTTP-host
    container.py        Container, Scope, Lifetime
    controller.py       OrderController, відображення JSON/DTO
tests/
  fixtures.py
  test_create_order.py
  test_domain.py
  test_container.py
  test_controller.py
examples/
  create_order.json
tools/
  check_architecture.py
  http_smoke.py
docs/
  contracts_lab3/
  clean_architecture_report.md
  test_run.txt
  isolated_test_run.txt
  architecture_check.txt
  demo_run.json.txt
  http_smoke_test.txt
```

`Order` є сутністю: має UUID; рівність і hash визначаються ідентичністю. `Money`, `Address`, `ProductSnapshot` та `OrderItem` є незмінними value objects, які порівнюються за значенням. UUID у ProductSnapshot посилається на товар іншого контексту; сам snapshot не має незалежного життєвого циклу в Order Service. Order є коренем агрегату і володіє tuple позицій замовлення.

Залежності коду: Application → Domain; Infrastructure → Domain; WebAPI → Application, Domain та Infrastructure. Конкретні адаптери імпортує лише композиційний корінь. У Domain немає HTTP, ORM, конфігурації або контейнера. Він використовує власні типи та стандартну бібліотеку Python.

Статична перевірка (architecture_check.txt) пройшла для 11 Python-файлів. Вона перевіряє абсолютні міжшарові імпорти та дозволений перелік імпортів Domain. Наявні відносні імпорти залишаються всередині шару; динамічних імпортів немає. Додатково окремий запуск сценарію (isolated_test_run.txt) підтвердив, що Infrastructure і WebAPI не імпортуються для його тестування.

2. Сценарій використання

Сценарій: `CreateOrder.execute(CreateOrderInput) → CreateOrderResult`.

Конструктор приймає три порти:

- `OrderRepository.add(order)` - запис агрегату.
- `Clock.now()` - час створення.
- `IdGenerator.next_id()` - ідентичність нового замовлення.

Вхідний DTO містить customer_id, tuple позицій з product_id/name/unit_price і quantity, а також shipping_address. DTO не залежить від HTTP/JSON. Алгоритм: перевірити клієнта й позиції → створити value objects адреси та товарів → отримати UUID і час через порти → створити Order → зберегти через OrderRepository → повернути DTO результату.

Доменні правила, узгоджені зі схемами ЛР №3:

- Від 1 до 100 позицій у запиті; quantity - ціле число від 1 до 1000.
- CustomerId і productId повинні бути UUID.
- Назва товару - від 1 до 200 символів, ціна - невід'ємна й скінченна.
- Місто - 2-100, вулиця - 2-200, індекс - 3-20, країна - 2-100 символів.
- Нове замовлення має статус NEW; час створення містить часовий пояс.
- Сума визначається як Σ(unitPrice × quantity), а не береться із запиту.

Нульова ціна допустима: оригінальний контракт має minimum: 0. Для текстів додатково відхиляються значення, що складаються лише з пробілів. Ціна і підсумок повинні вкладатися у скінченний double на зовнішній межі, оскільки OpenAPI/Proto3 використовують цей числовий формат.

Обчислення всередині Domain використовують Decimal, що уникає похибки `0.1 × 3` під час розрахунку. JSON-парсер зчитує дробові числа безпосередньо у Decimal. Перетворення на float виконується лише в контролері відповідно до вихідного контракту double; зовнішнє двійкове округлення можливе.

Приклад: Mechanical Keyboard, quantity=2, unitPrice=99.99 → totalAmount=199.98, status=NEW. Повний запит: create_order.json (../examples/create_order.json).

Відповідність ЛР №3: маршрут POST /orders; поля customerId, items, shippingAddress; відповідь id, customerId, items, totalAmount, status, shippingAddress, createdAt. CamelCase JSON перетворюється у snake_case DTO лише в WebAPI. Додаткові поля totalAmount/status від клієнта ігноруються та не можуть підмінити обчислену суму або початковий статус.

HTTP-відображення: 201 - успіх; 400 - неправильна структура/JSON; 422 - порушення доменного правила; 503 - помилка порту сховища. Помилки повертаються у форматі application/problem+json з type/title/status/detail. На HTTP-межі також перевірено 404, 413 і 415, у відповіді є заголовок кореляції з контракту ЛР №3. Коди 413/415/503 є явним розширенням поведінки навчального host відносно вихідного переліку відповідей POST.

Правила не розміщено у HTTP-обробнику: він лише читає JSON і передає керування контролеру. Сценарій не створює адаптери, не знає URL або БД. RepositoryError проходить через сценарій; лише контролер транслює його в 503.

3. Композиційний корінь

Єдине місце реєстрації - `build_container` у `src/WebAPI/bootstrap.py`. Навчальний контейнер підтримує Transient, Scoped та Singleton; фабрики передають залежності через конструктори.

Абстракція / ключ	Реалізація	Цикл	Обґрунтування
OrderRepository	InMemoryOrderRepository	Scoped	Один адаптер на запит; межа майбутнього unit of work
CreateOrder	CreateOrder	Scoped	Сценарій живе в тому ж scope, що й репозиторій
Clock	SystemClock	Singleton	Сервіс без змінного стану запиту
IdGenerator	UuidGenerator	Transient	Легкий незалежний генератор без стану
MemoryOrderStore	MemoryOrderStore	Singleton	Спільні дані процесу між запитами; доступ захищено Lock
OrderController	OrderController	Transient	Тонкий об'єкт без власного стану повторного використання

Поруч із кожною реєстрацією є коментар про вибір циклу. Для HTTP-запиту створюється окремий scope. Scoped-об'єкт повторно використовується в одному scope, але не між запитами. Singleton спільний, Transient новий за кожного resolve.

MemoryOrderStore містить дані агрегатів, а не поточного користувача/запиту. Його Scoped-реєстрація втрачала б замовлення після кожного запиту. Контейнер не дозволяє Singleton захопити Scoped-залежність, виявляє цикли, відхиляє дублікати реєстрацій та використання закритого scope.

Виклики resolve містяться лише в композиції WebAPI. Бізнес-код і контролер отримують готові залежності, тому Service Locator у бізнес-шарі відсутній. Контейнер навчальний: без автоматичного dispose і потокобезпечного resolve. Host використовує послідовний HTTPServer, що відповідає цим обмеженням.

4. Модульні тести

Використано `unittest` і `unittest.mock` зі стандартної бібліотеки Python. Тести одного сценарію містяться у `tests/test_create_order.py` і посилаються лише на Application, Domain та фікстури без зовнішнього вводу-виводу. Усі три порти замінено `Mock(spec=...)`.

Обов'язкові три випадки:

1. `test_success_saves_order_through_port` - перевіряє один запис через порт, UUID клієнта/замовлення, суму 199.98, адресу, час і статус NEW.
2. `test_empty_order_does_not_write` - порожнє замовлення відхиляється, жоден порт, включно з репозиторієм, не викликається.
3. `test_port_failure_propagates_without_hidden_io` - помилка Mock-репозиторію проходить через сценарій як той самий об'єкт RepositoryError.

Додаткові тести перевіряють кількість 0/1001/дробову/bool, від'ємну й нескінченну ціну, UUID, адресу, 100/101 позицію, нульову ціну, Decimal, переповнення суми, збій годинника, рівність сутностей і value objects, відображення DTO/ProblemDetails та життєві цикли DI.

Фактичний результат: 33 тести, OK. Повний лог (test_run.txt). Окремий прогін 12 тестів сценарію (isolated_test_run.txt) підтверджує: Infrastructure/WebAPI imports: [].

Модульний набір не відкриває БД, сокети або файли даних. Перевірки композиції виконуються в окремому тестовому модулі й використовують тільки пам'ять; WebAPI-host не запускається. HTTP smoke test (http_smoke_test.txt) окремо перевіряє реальні локальні запити, JSON, доменні відхилення і медіатип ProblemDetails. Він є інтеграційною перевіркою, а не частиною модульного набору.

Відтворення перевірок з кореня проєкту:

```
python -m unittest discover -s tests -v
python -m unittest tests.test_create_order -v
python tools/check_architecture.py
python -m src.WebAPI.bootstrap
git show lr4 --stat
```

5. Відповіді на контрольні питання

1. Чому Domain не повинен посилатися на Infrastructure? Domain містить правила замовлення, які не залежать від вибору сховища або транспорту. Залежність від ORM/HTTP прив'язує правила до технічних деталей. Порт OrderRepository належить внутрішньому шару, а адаптер реалізує цей порт зовні. Інший спосіб зберігання не змінює Order/CreateOrder.

2. Чим відрізняються Transient, Scoped і Singleton? Transient дає новий об'єкт щоразу; Scoped - один на область видимості; Singleton - один на контейнер/процес. Для запиту зручно об'єднати сценарій і репозиторій одним scope. Clock може бути Singleton, оскільки не має стану запиту. Захоплення Scoped-репозиторію Singleton-сценарієм є помилкою.

3. Як інтерфейс репозиторію дозволяє тестувати без БД? CreateOrder знає лише add(order). Mock з тим самим контрактом фіксує виклики й моделює помилки без реалізації сховища. Тест перевіряє правила та взаємодію з портом, не запускаючи БД або зовнішній сервіс.

Завдання на захист. Якщо змінити OrderRepository зі Scoped на Transient, тест `test_scoped_is_shared_within_request_and_distinct_across_requests` впаде на повторному вирішенні в одному scope. Якщо змінити на Singleton, він впаде при порівнянні різних scope. Новий порт можна додати як Protocol у внутрішній шар і параметр конструктора, передавши Mock у тесті без зміни чинного адаптера репозиторію.

6. Результат і межі реалізації

Реалізовано ядро Order Service для E-Commerce Cloud Platform, чотири шари, CreateOrder з constructor injection, in-memory адаптер, три DI-цикли, 33 успішних тести, звіт і локальний тег lr4 на адаптованому коміті. Контракт створення замовлення продовжує ЛР №3; його копії збережено для перевірки походження полів і правил.

ЛР №4 вимагає один сценарій: решта REST-операцій і gRPC-host тут не реалізуються. Proto3 залишається вихідним контрактом для подальшого адаптера. Автентифікацію/OIDC майбутнього API Gateway, платіжні операції, резервування Inventory та RabbitMQ цей сценарій не виконує. Ціна береться зі snapshot запиту відповідно до наявної моделі ЛР №3; довірене отримання ціни з каталогу є окремим майбутнім сценарієм інтеграції. Дані зберігаються тільки в пам'яті до завершення процесу; БД не піднімається.

ПІБ, група та URL віддаленого репозиторію залишені як ненадані дані.